NCUSCC 2026 秋季招新考核实践报告

一、项目概述

代码输出结果是在网页端的Linux容器里通过测试的，从最基础的命令行操作开始，逐步过渡到容器、多文件构建、指针内存问题、多线程排序等进阶内容。最开始我直接在网页终端里写代码，后来因为有时候容器申请使劲按很长，就决定在自己的电脑上用虚拟机搭建Linux环境，写完代码编译测试没问题，再同步到网页容器里提交，同时把所有代码按规范整理好传到自己的 GitHub 仓库里。

整个过程没有用任何图形化 IDE，所有代码都是在终端里用文本编辑器写的，靠g++编译、手动对比样例输出找问题，最终完成的

二、环境与工具准备

最开始是从装虚拟机开始的。我用的是VMware Workstation，在里面装了Ubuntu 22.04桌面版，选桌面版是因为系统自带 Firefox 浏览器，不用额外装就能直接登录GitHub网页操作。

系统装好之后，第一步就是在终端里装开发需要的基础工具。执行了两条命令：先sudo apt update更新软件源，然后sudo apt install -y build-essential git cmake nano。这里面 build-essential 带了g++ 编译器和make构建工具，git用来做版本管理，cmake用来处理多文件项目，nano就是我全程用的文本编辑器。装完之后我分别跑了g++ --version、git --version确认工具都能正常运行。

接下来配置git的身份信息，因为第一次 commit 会报错说不知道作者是谁，我执行了 git config --global user.email写上自己的邮箱，git config --global user.name写上自己的GitHub用户名。然后在 GitHub网页上新建了一个叫ncuscc-practice的仓库，把本地代码和这个远程仓库关联起来。

整理一下我用到的所有工具：
1.虚拟化平台是VMware Workstation
2.操作系统是Ubuntu 22.04，所有开发都在这个系统里完成
3.编辑器是nano，终端里直接打开就能写，不需要切到图形界面
4.编译器是g++
5.版本管理用git，配合GitHub远程仓库存所有代码

开发过程中最常用的终端命令有这些：
1.建文件夹用 mkdir 关卡名，每个关卡单独一个目录
2.打开文件写代码用 nano 文件名，写完按 Ctrl+O 保存、Ctrl+X 退出
3.编译单文件程序用 g++ -std=c++17 -Wall -Wextra -o main main.cpp
4.运行程序测试用 ./main，要对照样例就用 ./main < 样例输入.in
5.看目录下有什么文件用 ls，看路径用 pwd
6.同步代码到 GitHub 用 git add .、git commit -m "说明"、git push 三步

三、仓库目录结构（最终 GitHub 存储规范）
所有代码和文档统一放在 ncuscc-practice 仓库里，结构 下：

```
ncuscc-practice/
├── src/                    # 所有项目源码都放在这个目录下
│   ├── 01-读数统计/
│   │   └── main.cpp
│   ├── 02-设备巡检/
│   │   └── main.cpp
│   ├── 03-环形窗口/
│   │   └── main.cpp
│   ├── 04-多文件统计/
│   │   ├── CMakeLists.txt
│   │   ├── main.cpp
│   │   └── format.cpp
│   ├── 05-读数变化/
```

```
│   │   └── main.cpp
│   ├── 06-馆藏目录/
│   │   └── main.c
│   ├── 07-借阅台账/
│   │   └── main.cpp
│   └── 08-四线程快排/
│       ├── main.cpp
│       └── data.txt
├── README.md                # 项目基本介绍
└── NCUSCC实践报告.pdf       # 技术报告和个人感悟，导出PDF
```

四、实践过程与设计细节

最开始是基础关，主要是熟悉Linux和git/github基础操作，包括目录操作、find 查找、管道这些命令等。比 第一关要在 /workspace 里找到密钥文件，调用系统提供的exam命令生成flag，主要是练怎么看命令的帮助信息，用 --help 参数搞清楚参数怎么传

然后就是C++写程序，改bug了

第一个C++题是读数统计，输入一堆整数，输出最大、最小、平均值。我把min和max都初始化成 0，第一个数读进来的时候直接赋值给它们，后面再逐个比较，比max大的成为新的max，比min小的成为新的min，结果正确

第二道是设备巡检记录，要统计不同设备的故障次数和运行时长。这里用了map容器，把设备编号对应到两个计数器上，每读一条记录就更新对应的计数，最后遍历map输出结果就好

第三道是固定容量读数窗口，要求保留最近8个读数，支持求和、平均、最值查询。这里用了环形数组的思路，数组大小固定 8 个，每次新进来一个数就把最老的那个覆盖掉，用一个指针记录下一个要覆盖的位置，这样不管多少个数进来，都不用动态扩容

第四道是多文件统计工具，要求把代码拆成多个.cpp文件，用CMake构建。最开始写CMakeLists 的时候漏了 format.cpp这个源文件，结果链接的时候报一堆 undefined reference 的错，后来把源文件都加到 add_executable 列表里就好了

第五道是读数变化观察，要统计上升、下降、平稳的次数。这里最开始把第一个数也拿去和"前一个数"比，结果多算了一次，后来改成从第二个数开始，每个数和前一个比较，结果就对了。

第六道是C语言的馆藏目录卡题，主要是练指针和缓冲区。最开始写的时候指针移动会把后面的内容覆盖了，两遍输出都成了相同的内容，修改方式是将char shared[**9**]改成二维度数组，这样就能分别打印了

第七道是借阅台账，模拟图书馆借书还书。这里踩的坑最多：最开始以为borrow一次只借一本，后来读样例才发现 borrow后面也跟了数字，是一次借多本，最开始漏读这个数字导致整个输入流都错位了，后面所有操作读出来都是乱的。还有个问题是，只要有一条操作被拒绝了（比 借的书不够、还没借过的书），就要输出rejected，而且最后不能再输出那几行统计信息，最开始我不管有没有错误都输出统计，结果和样例对不上，加了个错误判断，有错误的时候就跳过最后的统计输出，就和样例一致了

最后一道是四线程快速排序观察题，这里用了C++标准库的thread，写了个带线程数限制的快速排序：用一个全局计数器加互斥锁，保证同时跑的线程不超过4个，当区间小于4个元素的时候就不再拆线程，直接串行排序，所有子线程跑完之后再 join汇合。因为沙盒里只有一个CPU，开多线程其实没有真正的并行加速，报告里也写了这一点，这部分的内容是现学的，所以掌握的还不熟练，做的结果可能也有欠缺，还需要继续学习

五、遇到的问题与解决方法

整个过程踩了很多小坑，大部分都是刚开始不熟悉命令行环境导致的，整理一下印象最深的几个：
最开始用sudo的时候密码输错了，终端卡住不动，不知道怎么退出去，后来知道按Ctrl+C就能取消当前命令重新来，以及不知道输入密码时密码是不显示的，输入时看到输入光标一直不动，问AI才知道输入不显示

nano打开文件之后不知道怎么保存退出，最开始直接关终端，结果写的东西全没了，后来记住了Ctrl+O 保存、回车确认文件名、再按Ctrl+X退出这个流程。

最开始在错误的目录下跑g++，一直报"main.cpp 不存在"的错，后来养成习惯，每次编译前先ls一下看看当前目录有什么文件，确认main.cpp在这再编译。

第一次git commit的时候直接报错，说作者身份不知道，要自己配置邮箱和用户名。第一次使用git push，要输入 password，问AI然后知道是从GitHub里面复制Token来输入，再问AI，把远程仓库改成ssh格式，这样就不用每次都输入 password了

六、个人感悟

这次从头到尾做下来，最大的感受是命令行环境虽然看起来不直观，但用熟了其实效率很高。最开始我连进哪个目录都要ls半天，现在已经能熟练地建文件夹、写代码、编译、测试、提交一套流程走下来了。之前写 C++ 都是在 Visual Studio 里，点一下按钮就编译运行，出了问题也是要打断点调试。这次所有东西都在终端里做，没有图形化的提示，才真正搞清楚编译、链接这些步骤到底是怎么回事

还有就是版本控制的习惯，以前写代码就是改坏了就删了重写，现在做完一个版本就commit一次，出了问题随时能退回到上一个版本，代码都存在 GitHub 上，换个电脑也能接着写。

总的来说，这次不只是做了几道编程题，更重要的是熟悉了Linux下的开发流程，了解了使用GitHub远程仓库管理代码，从本地git保存、提交代码再到上传到GitHub上，中途遇到问题和不懂的地方就去网上搜索相关的资料并向AI发问，最终也是磕磕绊绊地完成了相应的内容。这次完成考核内容也锻炼了我的自学能力和解决问题的能力，对我的成长很有意义